

-  Secrets
-  ABAP
-  Ansible
-  Apex
-  AzureResourceManager
-  C
-  C#
-  C++
-  CloudFormation
-  COBOL
-  CSS
-  **Dart**
-  Docker
-  Flex
-  Go
-  HTML
-  Java
-  JavaScript
-  JCL
-  Kotlin
-  Kubernetes
-  Objective C
-  PHP
-  PL/I
-  PL/SQL
-  Python
-  RPG
-  Ruby
-  Scala
-  Swift
-  Terraform
-  Text
-  TypeScript
-  T-SQL
-  VB.NET
-  VB6
-  XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules **115**

 Bug **15**

 Code Smell **100**

Tags ▾

Impact ▾

Clean code attribute ▾

Search by name... 🔍

Null-aware operators should be preferred
 Code Smell
Generic function type aliases should be preferred
 Code Smell
Type parameters should not shadow other type parameters
 Code Smell
"static final" declarations should be "const" instead
 Code Smell
Interpolation should be used instead of String concatenation
 Code Smell
Collection literals should be preferred
 Code Smell
Conditional assignment should be preferred
 Code Smell
"this" should only be used when required
 Code Smell
Exceptions should not be ignored
 Code Smell
Code annotated as deprecated should not be used
 Code Smell
File names should comply with a naming convention
 Code Smell
Unused local variables should be removed

Null-aware operators should be preferred

Analyze your code

Intentionality - Clear

Maintainability ⬇️

 Code Smell

 Minor ⓘ

Why is this an issue?

How can I fix it?

More Info

`null`-aware operators allow to safely access nested properties or methods of an object without having to check for the existence of each intermediate property manually. It provides a concise and safe way to access nested properties or methods without having to write complex and error-prone `null` checks.

This rule flags logical operations that can be safely replaced with the `? .` `null`-aware operator.

Available In:

sonarcloud 

